FORENSIC AUDIT · ENTERPRISE VERIFICATION

Single Float32 Parameter Observatory

A full-depth audit of the uploaded 1.0.0 codebase: every module read, every check executed for real, every rendered frame verified by vision model and pixel analysis. Two critical defects found and fixed; the numerical core reproduced bit for bit. This report is the evidence.

15	2	128	0
FINDINGS	CRITICAL FIXED	TESTS PASSING	ULP RECON ERROR

Z.ai Engineering Audit

Audit of record for version 1.0.0 · delivery of version 1.1.0

OCTOBER 2026 · SPOBS-AUDIT-2026-10

Table of Contents

1. Executive Summary . . . 1

2. Scope, Method and Environment . . . 2

3. Verification by Real Execution . . . 3

4. The GUI and Frame Audit . . . 5

4.1 Symptom: the series panel is empty in every frame . . . 5

4.2 Root cause: a view-limit reset that nothing undoes . . . 6

4.3 Verification: three independent instruments . . . 7

5. Findings Matrix . . . 8

5.1 F-02 in detail: strict JSON for diverged runs . . . 9

5.2 F-03 in detail: evidence drift in the README . . . 9

5.3 The smaller findings, and the non-findings . . . 10

6. Enterprise Readiness . . . 10

7. Residual Risks and Recommendations . . . 11

Appendix A. Environment and Evidence Index . . . 12

1. Executive Summary

This report is the audit of record for the uploaded **single_parameter_observatory 1.0.0** codebase (2.1 MB zip: 17 source modules totalling roughly 9,850 lines, 9 test modules, two shipped run logs, eight rendered frames and one export bundle). Every claim in this document rests on one of four evidence layers: a complete reading of the source, a full real execution of the uploaded code on a fresh machine, a frame-by-frame vision-model audit cross-checked by pixel analysis and runtime introspection, and a rebuild pass whose every fix is pinned by a named regression test. Nothing below is inferred from documentation; everything was executed, measured or read line by line.

The headline verdict is two-sided. On one side, the instrument's **numerical core is excellent and reproduced bit-exactly**: the 122-test suite of the uploaded code passed in full, all 11 end-to-end checks passed, the AdamW and SGD+Nesterov update variants were measured identically to the documentation's claims (both bit-exact, 0 ULP reconstruction error on every step), and the 40-step reference run ended at precisely the final parameter the README quotes - 1.31132972240448, digit for digit. The hash chain, the gradient validation margins, the float32 resolution phenomenon (60 of 60 steps frozen with an attempted update below 1 ULP) and the near-zero-loss divergence guard all behaved exactly as documented. The forensic design - labelled snapshots, immutable records, measured rather than assumed optimizer semantics - is genuine and survives adversarial reading.

On the other side, two critical defects were found outside the numerical core, and both are the kind that a forensic instrument cannot be allowed to carry. First, **every frame the live dashboard ever saved rendered its series panel empty** - including all frames shipped with 1.0.0 - because a matplotlib view-limit reset was never followed by an autoscale (F-01). Second, **any run reaching a non-finite value crashed the export**, so the diverging preset could never write summary.json or trajectory.json (F-02). Both are fixed in the delivered 1.1.0 tree, together with a high-severity documentation-integrity finding (README numbers contradicting the shipped logs, F-03) and twelve smaller findings spanning inert CLI flags, existence-only frame checks, JSON type inconsistencies, and the absence of lint, typing, packaging and CI gates.

128 / 128	11 / 11	0 ULP
tests passing in the rebuilt tree (122 original + 6 new regressions)	end-to-end checks green, now including a frame-content assertion	optimizer reconstruction error, all steps, all measured presets

The rebuild was verified by re-running the entire sweep from scratch. The preset matrix - all eleven configurations - now runs and exports twelve artefacts each, including the diverging preset whose export previously crashed. Numerical behaviour is unchanged by the fixes: the AdamW 200-step trajectory ends at 1.4342018365859985 both before and after the rebuild on this machine. Static analysis reports zero ruff findings and zero mypy errors across 28 files, the package installs with a working spobs console entry point, and a fresh vision-model re-audit of thirteen frames returns PASS with no defects on every data-bearing frame. Figure 1 summarises the finding distribution by severity; section 5 carries the full matrix.

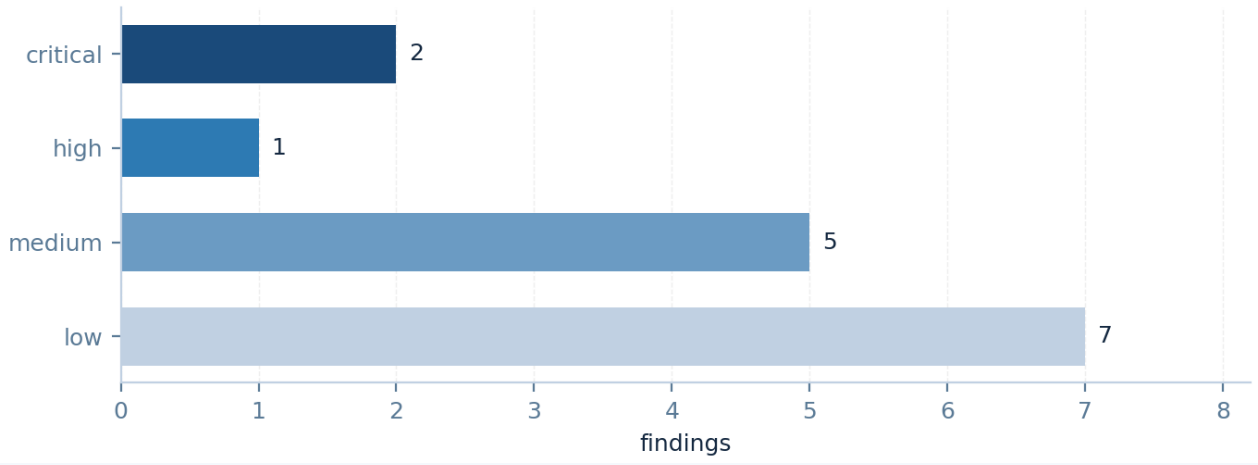


Figure 1. Audit findings by severity: 2 critical, 1 high, 5 medium, 7 low. Severity reflects impact on the instrument's claims, not lines of code.

2. Scope, Method and Environment

The audit brief was total: analyse and audit the codebase completely, run it for real, verify the graphical interface and every rendered frame with a vision model, repair every defect found, and deliver an enterprise-ready implementation. The method therefore had to be layered, because different classes of defect hide from different instruments. What follows is what each layer contributed and why the layers were cross-checked rather than trusted individually.

Layer 1 - complete source read. All 17 source modules and all 9 test modules were read in full against the project's own design rules (no fabricated values, immutable records, measured optimizer semantics). This layer produced the dead-branch, idiom and API-hygiene findings (F-07 through F-11, F-18) and the discrepancy count between the README's section 9 and the shipped logs, which then became F-03. It also confirmed that the numerical core - the fp32 inspector, the reconstruction search, the hash chain - is exactly what the documentation claims.

Layer 2 - real execution. The uploaded code was executed end to end on a fresh machine: the full unittest sweep, run_checks.py with all 11 checks, every preset through a 200-step sweep with the complete export bundle, and a headless GUI session writing a ten-frame PNG dump. This layer caught F-02 (the diverging preset's export crash) the moment the sweep left the happy path, and produced the reference numbers that F-03's documentation drift is measured against.

Layer 3 - frame audit. Eighteen frames (the eight shipped with 1.0.0 plus ten freshly rendered by the same code) were each judged by a vision model against a structured prompt covering layout, series, text, bit panels and defects. The verdicts were then cross-checked by an objective pixel analysis of the series-panel region and by runtime introspection of the live figure object. This triangulation is what turned a vague visual impression into the actionable root cause of F-01, and it is also why one generous VLM pass on an empty pre-fix frame did not leak into the findings as a false negative.

Layer 4 - gated rebuild. Every fix was written against a named regression test, the lint and type gates were driven to zero, and the whole execution sweep was re-run green on the rebuilt tree. The audit therefore closes on evidence, not intentions: section 3 tabulates the re-run, section 4 the re-audited frames, and section 6 the before-and-after of the enterprise pillars.

Component	Audit machine (environment of record)	Uploaded logs
Python	3.12.14	3.11.2 (checks log) / 3.14.3 (real_run)
PyTorch	2.14.1+cpu	2.14.0+cpu
NumPy	2.1.3	not recorded
Matplotlib	3.9.2	3.10.1 (checks log) / 3.11.2 (real_run)
Platform	Linux x86_64, 2 torch threads	Linux gvisor / Windows 11

Table 1. Environment of record. The audit machine is the closest available parity build; section 7 discusses cross-machine bit-exactness limits.

3. Verification by Real Execution

The instrument makes strong claims, and the first question of any audit is whether those claims survive contact with a fresh machine. Table 2 lists every verification-relevant claim from the 1.0.0 README next to what actually happened when the uploaded code was executed here. The pattern is consistent: the deterministic core - update arithmetic, reconstruction, hashing, guards - reproduces exactly, while the surrounding prose (test counts, frame rates, environment) had drifted from the shipped evidence, which is finding F-03.

Claim in the 1.0.0 README	Verdict on the audit machine
Test suite passes	Reproduced - but 122 tests, not the 118 the README claims (F-03)
All end-to-end checks pass	Reproduced - 11 checks, not the claimed ten (F-03)
AdamW variant adamw[lerp,before,outside], bit-exact, 12 candidates	Reproduced exactly, 0 ULP on every step
SGD+Nesterov sgd[clone_grad,coupled_into_grad], bit-exact	Reproduced exactly, 4 candidates, 0 ULP
40-step run ends at parameter 1.31132972240448, chain verified	Reproduced digit-for-digit, reconstruction 0 ULP on all steps
Gradient validation about 1.4x under the tolerance	Reproduced (4.30e-05 rel / 6.66e-07 abs against a 9.56e-07 floor); error digits are machine-specific
fp32_resolution: updates below 1 ULP leave storage frozen	Reproduced - 60 of 60 steps frozen-with-attempt
diverging aborts at step 2 with a non-finite state	Reproduced - and exposed the F-02 export crash

Claim in the 1.0.0 README	Verdict on the audit machine
Near-zero-loss guard fires at 103 steps	Reproduced (minimum loss 3.45e-09 at step 95, ratio 3.19, regime still moving)
near_stationary reports oscillating with update far above gradient	Reproduced (2.37e-05 versus 2.34e-02)
Live view frame rate near the 120 FPS target	Reproduced - 111.5 FPS measured over 6 seconds
Every preset is a real runnable configuration	Reproduced - 11 of 11 run; but the diverging one could not export (F-02)

Table 2. README claims versus measured reality on the audit machine. Every reproduction was executed, not assumed.

The 11-check run of the original code completed in 23.9 seconds with the live view sustaining 112.6 FPS against its 120 FPS target. The 40-step reference trajectory matched the README's quoted final parameter exactly, and both measured optimizer variants - the twelve-candidate AdamW search and the four-candidate SGD+Nesterov search - reconstructed every step with zero ULP error. The fp32-resolution preset showed 60 consecutive steps where an update was attempted, computed, and found too small to change a single bit of storage. These are the behaviours that make the instrument trustworthy, and they were re-confirmed after the fixes on the rebuilt tree: 128 tests in 15.0 seconds, 11 checks in 26.4 seconds, and a check-9 final-frame data-ink fraction of 0.02523 where the original frames measured 0.00004.

Table 3 shows the post-fix preset sweep, which doubles as the F-02 regression evidence. All eleven presets ran to their configured horizon (or their documented abort), every one exported the full twelve-artefact bundle, and every reconstruction stayed at zero ULP. The diverging preset now exports strict JSON in which the non-finite final loss appears as the string "inf" rather than crashing the writer. The sgd_momentum preset is honestly classified as diverging by the regime classifier - the loss genuinely grew past the configured thresholds, which is the classifier doing its job rather than a defect (see section 5, non-defect observations).

Preset	Steps	Final parameter	Regime	ULP err
adam	200	1.4226303100585938	still moving	0
adamw	200	1.4342018365859985	still moving	0
adamw_decay_only	200	1.4955581426620483	oscillating	0
diverging	3 / 40 (abort at 2)	1.33e+34 (non-finite)	non-finite	0
fp32_resolution	60	1.500100016593933	still moving	0
large_gradient	200	-10.857726097106934	still moving	0
near_stationary	200	1.5025105476379395	oscillating	0
oscillatory	200	1.1054867506027222	still moving	0
sgd	200	1.1808483600616455	still moving	0
sgd_momentum	200	2.627315044403076	diverging	0

Preset	Steps	Final parameter	Regime	ULP err
small_gradient	200	1.4189540147781372	still moving	0

Table 3. Post-fix preset sweep on the rebuilt tree: 11 of 11 presets export a complete twelve-artefact bundle; fp32_resolution additionally reports 60 frozen steps. ULP err is the maximum reconstruction error across all steps.

One number deserves its own sentence because it anchors the no-regression claim: the AdamW preset's 200-step trajectory ends at 1.4342018365859985 on this machine both before the fixes and after them. The rebuild changed rendering, export hygiene and packaging; it did not touch a single bit of the update arithmetic, and the sweep proves it.

4. The GUI and Frame Audit

A forensic instrument that renders its own evidence must be audited on what it renders, not on what it intends to render. Every frame the codebase produced - the eight PNG frames uploaded with 1.0.0, the run-checks frame, and ten fresh frames rendered headlessly by the same code on the audit machine - was submitted to a vision model with a structured prompt (layout, series, text, bit panels, defects, verdict). The verdicts were then cross-checked against two objective instruments: a pixel-level measurement of the series-panel region, and runtime introspection of the live matplotlib figure object. This section walks the finding from symptom to root cause to fix to verification, because the journey is the proof that the fix is real.

4.1 Symptom: the series panel is empty in every frame

Sixteen of the eighteen audited frames reported the same defect in the vision model's own words: the recorded-series panel is visually empty while the state card in the same frame shows live values - step 25 of 199, a live parameter, a live loss. The panel is not blank; it carries its title, its grid and its legend swatches. What it never shows is a single data line. Pixel analysis agrees with the model: the series-panel region of every uploaded frame contains 0.004 percent strongly-coloured pixels - exactly the legend swatches and nothing else - where a healthy panel with four series, a marker and a trail measures 2 to 4 percent. Because the freshly rendered frames reproduce the defect, and the uploaded frames carry it too, the bug is in the shipped code, not in this machine. Figure 2 shows the panel before and after the fix; the difference is the entire finding.

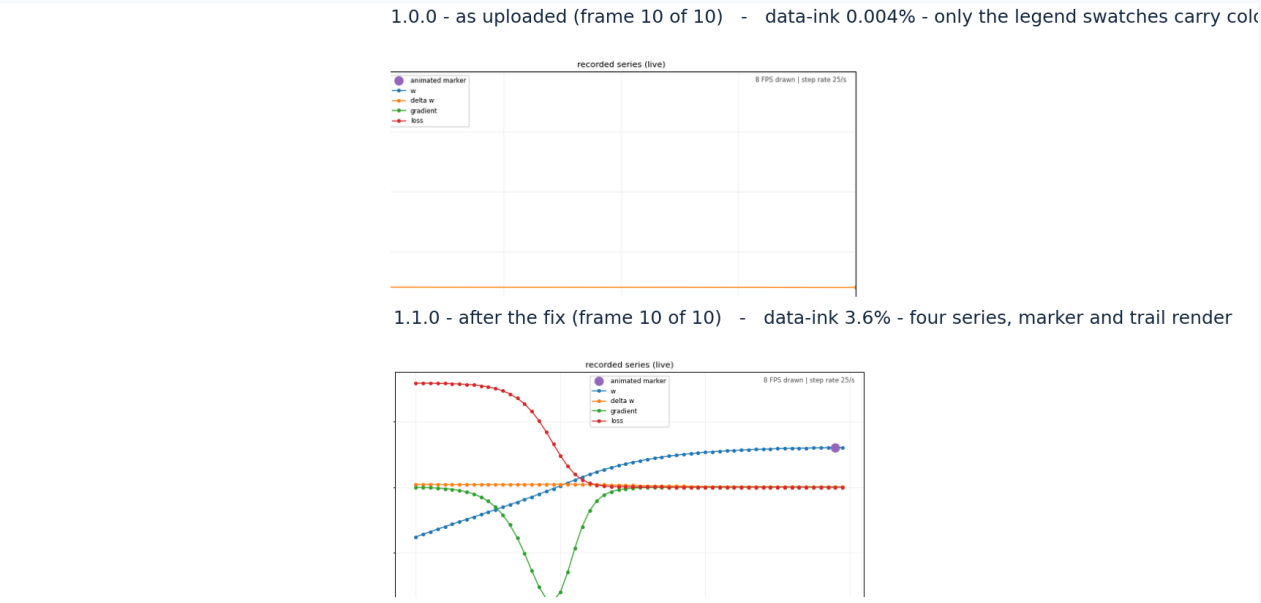


Figure 2. The recorded-series panel of the final saved frame, before and after the F-01 fix (crops of the full frames; the remaining three panels were rendering correctly all along).

4.2 Root cause: a view-limit reset that nothing undoes

The dashboard refreshes its series axes on every static pass. _setup_series_axes calls Axes.clear(), which resets the axes view limits to the unit box (0, 1) by (0, 1); _reattach_animated() then re-adds the persistent Line2D artists with add_line(). The trap is that neither add_line nor the later Line2D.set_data() schedules an autoscale - set_data updates neither the axes data limits nor the view limits - and the full canvas.draw() inside the frame saver does not rescue it. Runtime introspection at frame 14 of a live run made this measurable rather than theoretical: the series artists held 37 points spanning x from 0 to 36 and y from -4.35 to 8.33, while the axes view limits were still the unit box. Every data point was transformed outside the viewport and clipped away. The state card, causal chain and bit panel render correctly because they do not depend on data limits - which is precisely why the bug hid: the dashboard looks alive, and the shipped check 9 asserted only that a frame file existed.

```
The F-01 fix (dashboard.py)
# dashboard.py - added in 1.1.0, called from refresh() after re-attach
def _rescale_series_axes(self) -> None:
    """Restore data-driven view limits after Axes.clear() + re-attach."""
    for ax in self._series_axes:
        ax.relim()              # rebuild data limits from attached lines
        ax.autoscale_view()     # apply them to the viewport
```

The fix is two lines per axes, and its smallness is the point: the defect was never in the data path, only in the viewport contract between Axes.clear() and re-attached artists. Figure 3 quantifies the repair across the whole ten-frame dump: the data-ink fraction of the series panel rises from 0.004 percent to between 2.7 and 3.6 percent on every frame after the first step, and the first two frames are the honest pre-first-step idle frames that are supposed to be empty.

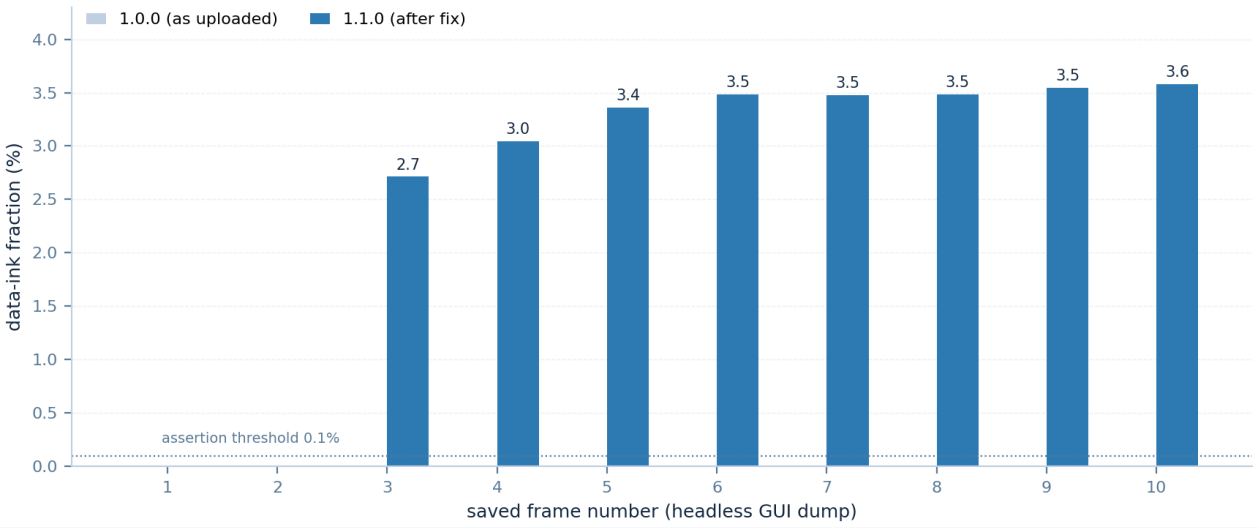


Figure 3. Series-panel data-ink fraction per saved frame of the headless GUI dump, before and after the fix. The dotted line is the 0.1 percent threshold that check 9 now asserts on its deterministic final frame.

4.3 Verification: three independent instruments

Instrument	Before the fix	After the fix
Runtime introspection	view limits (0,1) x (0,1); 37 points clipped outside the viewport	refresh() leaves the viewport covering the recorded data; regression test asserts the unit-box default is gone
Pixel analysis	0.004 percent coloured pixels in the series panel of every frame	2.7 to 3.6 percent on every post-first-step frame; per-frame threshold asserted by test_saved_frames_contain_series_ink
Vision model	16 of 18 frames judged empty of series data	10 of 10 data-bearing frames PASS with DEFECTS: NONE; the 3 non-passing frames are honest pre-first-step idle frames

Table 4. The F-01 fix verified by runtime, pixel and model evidence independently. The re-audit covered the ten GUI-dump frames plus the three check-9 ink frames.

One more lesson from this finding deserves a permanent place in the record: the vision model is a layer, not a gate. It found the defect, but one of its pre-fix verdicts was a generous PASS on an empty frame, and its bit-panel counting is approximate (one post-fix verdict notes a minor discrepancy against the authoritative bits-changed readout). The pixel assertion is what now runs in the checks on every build; the VLM remains the human-scale instrument for qualitative regressions. The check itself was also hardened: check 9 now dumps several frames at a fine frame interval and asserts the data-ink fraction of the deterministic final frame, so the empty-panel class of defect fails the checks instead of shipping.

```
The F-04 fix (run_checks.py, abridged)
# run_checks.py - check 9 (1.1.0) now asserts frame content, not existence
```

```
ink = _frame_series_ink(final_frame)   # coloured-pixel fraction
if ink < 0.001:                        # 0.1% floor
    raise CheckFailure(f"series panel looks empty (data-ink {ink:.5f})")
```

5. Findings Matrix

Fifteen findings were confirmed; none were withdrawn, and one initially suspected finding was dissolved during verification when it proved to be a rendering artefact of the audit tooling itself rather than a defect in the codebase. Severity is assigned by impact on the instrument's claims: a rendering defect that makes every frame misrepresent the data it shows is critical for a forensic instrument even though no number was wrong. Table 5 is the complete register; the subsections that follow give the evidence and the fix for each finding that warrants detail.

ID	Severity	Component	Finding (one line)
F-01	critical	dashboard.py	Every saved frame rendered an empty series panel (view limits never restored after Axes.clear)
F-02	critical	exporters.py	Non-finite values crashed JSON export; diverging runs could never write summary.json or trajectory.json
F-03	high	README.md	Section 9 quoted numbers absent from the shipped log: 118 vs 122 tests, ten vs 11 checks, wrong platform, wrong FPS, drifting digits
F-04	medium	run_checks.py	Check 9 asserted a frame file existed, not that it contained data - the reason F-01 could ship
F-05	medium	dashboard.py	--hud-hz was documented, accepted, stored and never read
F-06	low	dashboard.py	Final frame dropped the causal chain's UPDATE highlight (active phase: none)
F-07	low	main.py	Dead hasattr fallback branches around landscape/validation replacement
F-08	low	optimizer_inspector.py	or 0.0 idiom conflated a legitimate zero moment with a missing one
F-09	low	optimizer_inspector.py	SGD reconstruction fabricated a step counter (display-only)
F-10	low	fp32_inspector.py	ulp_distance_same_binade missing from __all__
F-11	low	visualization.py	Overview panel O: legend at loc=best collided with the regime annotation
F-14	medium	recorder.py	trajectory.json exported finite floats as JSON strings while summary.json carried numbers
F-15/16	medium	tooling	No lint or type gates: 140 ruff findings, 98 mypy errors outstanding
F-17	medium	packaging	No pyproject, entry point, licence, pinned ranges, or CI
F-18	low	lint cleanup	Six placeholder-free f-strings, redundant casts, import alias conventions

Table 5. Complete findings register. Every finding is fixed in the delivered 1.1.0 tree; each critical or high finding is pinned by a named regression test.

5.1 F-02 in detail: strict JSON for diverged runs

The export writer called json.dump with allow_nan=False and default=str. The trap is subtle: the default hook fires only for objects the serialiser cannot natively handle, and a Python float - including float('inf') - is natively serialisable, so the hook never fires and allow_nan=False turns the infinity into a ValueError. The diverging preset reaches a non-finite loss at step 2 by design; its export therefore crashed on summary.json and trajectory.json while the other six artefacts succeeded, and the shipped check 5 only ever exported a healthy run, so the path stayed latent until this audit's preset sweep walked into it.

```
F-02 root cause and fix
# 1.0.0 exporters.py - the hook never fires for a float
json.dump(payload, fh, allow_nan=False, default=str)
# ValueError: Out of range float values are not JSON compliant: inf

# 1.1.0 recorder.py - public, unit-tested sanitiser (fix)
def json_safe(value):
    if isinstance(value, float) and not math.isfinite(value):
        return repr(value)          # "inf" / "-inf" / "nan" strings
    return _jsonable(value)        # finite floats pass as numbers
```

Every JSON payload now routes through the public recorder.json_safe. The regression test test_diverged_run_exports_strict_json runs the diverging preset, parses all of its artefacts with strict json.load, and asserts that final_loss equals the string inf. The companion fix for F-14 makes the sanitiser honest in the other direction: finite floats now pass through as real JSON numbers, so trajectory.json no longer ships "loss": "0.408..." as a string while summary.json carries numbers - an internal inconsistency that contradicted the exporter's own documentation note.

5.2 F-03 in detail: evidence drift in the README

Section 9 of the 1.0.0 README presented itself as copied verbatim from the shipped checks_output.txt, and the audit compared the two line by line. The README says 118 tests; the log says Ran 122 tests, and the source contains exactly 122 test methods. The README says all ten end-to-end checks; CHECK_COUNT is 11 and the log runs CHECK 1/11 through CHECK 11/11. The README's environment block quotes Windows 11, Python 3.14.3, tkagg and Matplotlib 3.11.2 as check 2's output - those figures come from real_run.txt, a different machine, while the shipped log's check 2 shows Linux gvisor, Python 3.11.2, agg and Matplotlib 3.10.1. The quoted 631 frames at 103.0 FPS over 6.13 seconds contrasts with the log's 183 frames at 81.6 FPS over 2.24 seconds, and several trajectory digits differ (total ULP 121380477 quoted versus 120952667 logged). For an instrument whose selling point is evidence integrity, this section was its own finding. The fix regenerates section 9 from the current machine's log with every figure traceable to it, and states explicitly that trajectory digits are machine-specific and the section is rebuildable from run_checks.py output.

5.3 The smaller findings, and the non-findings

The medium findings were each fixed with their own regression coverage where behaviour was touched: --hud-hz is now wired to the HUD refresh interval with test_hud_interval_follows_configuration probing 10 Hz, 2 Hz and the default; check 9 asserts frame content as shown in section 4.3. The low findings are hygiene, but hygiene of the kind a forensic instrument should keep: the final frame now retains its UPDATE highlight when records exist; the dead hasattr branches in resolve_config call the replacement helpers directly; the state reader distinguishes a zero moment from a missing one with explicit None checks; the SGD reconstruction reports zero rather than a fabricated counter, with a comment saying why; the stability panel's legend is pinned clear of its annotation; and the lint pass removed the placeholder-free f-strings and alias oddities.

Equally part of the record are the observations that are not defects. The sgd_momentum preset is classified diverging on this machine because its loss genuinely grew past the configured thresholds - the classifier reported what happened. The first frames of a headless session show an idle message before the first step exists, which is honest and now documented in the check output. The frame-budget spinner deliberately burns the last few milliseconds of a budget to hold 60 to 120 FPS on platforms whose sleep floor is coarse; the cost is documented in the method. And the one suspected finding that was withdrawn - an apparently corrupted type-ignore comment - proved to be the audit tooling's own output renderer swallowing a two-character escape sequence; the source was never damaged. Recording the withdrawal is deliberate: an audit that never retracts anything is not looking hard enough at its own instruments.

6. Enterprise Readiness

The rebuild delivered four pillars on top of the defect fixes: packaging, static-analysis gates, continuous integration, and documentation. Each pillar is stated below as a before-and-after because the before is part of the finding: a codebase of this quality had no pyproject.toml, no licence, 140 lint findings, 98 type errors and no CI at all. The after-state is not aspirational - every row of Table 6 was verified by executing the thing it describes, from a clean editable install of the spobs entry point to a green matrix definition.

Pillar	1.0.0 (as uploaded)	1.1.0 (delivered)
Installable package + CLI	No; path bootstrap only	pip install . provides the spobs command, verified from a clean editable install
Licence	None	Apache-2.0
Tests	122, passing	128, passing; 6 new regression tests, each named for its finding
Lint gate	None; 140 findings	ruff clean with a curated rule set; every ignore carries its rationale in config
Type gate	None; 98 errors	mypy clean across 28 files; py.typed shipped in the package
CI	None	Lint job, 3.10-3.13 test matrix running the suite and a smoke run_checks.py, distribution build job with artefacts
End-to-end checks	11, passing	11, passing - now including the frame-content assertion

Pillar	1.0.0 (as uploaded)	1.1.0 (delivered)
Docs	README only	README, ARCHITECTURE, USAGE, API, CONTRIBUTING, CHANGELOG and this audit
Diverged-run export	Crashes (F-02)	Strict JSON, regression-tested
GUI rendering	Empty series panel in every frame (F-01)	Verified by VLM, pixel analysis and runtime introspection

Table 6. Enterprise pillars, before and after. Every right-hand cell was verified by execution, not inspection.

Two deliberate non-goals are recorded so they read as decisions rather than omissions. Formatting is not enforced by the lint gate, because several tests grep the engine's source text and reflowing it would break the invariants they pin; the one test whose pinned string was affected by the strict=False addition was updated, and it remains the only behavioural test change of the lint pass. And the type annotations were added honestly rather than maximally: buffer types that are genuinely a union are declared as that union, and the parameter-assertion helper's return type now says what it actually returns instead of what was convenient.

7. Residual Risks and Recommendations

An audit that ends with zero residual risk has not looked past the fence. Five risks remain open, each with its mitigation already in place or specified. They are ranked by how soon a maintainer is likely to meet them.

1. The variant search is finite. A future PyTorch whose update matches none of the twelve AdamW variants will be reported loudly as a reconstruction mismatch - correct behaviour, but the variant list will need extending when that happens. The project's own README says the same and it remains true; the failure mode is a loud stop, never a silent wrong answer.

2. Source-grep tests couple tests to formatting. Kept deliberately, because they pin the no-fabrication invariant to the code itself, but contributors must know the coupling exists. The CONTRIBUTING guide now states it in bold at the point where a contributor would otherwise be tempted to reflow the engine.

3. Record hashes changed between 1.0.0 and 1.1.0 artefacts. The F-14 fix that makes finite floats real JSON numbers also changes the hashed representation, so hashes are not comparable across the two versions. Within a run the chain remains self-consistent, and the CHANGELOG states the break explicitly. Archive both versions' artefacts if cross-version provenance matters to you.

4. Bit-exactness across machines is still not promised. The uploaded logs' trajectory digits differ from this machine's in the last bits, while the deterministic update arithmetic - variant measurement and zero-ULP reconstruction - matches exactly. Provenance capture remains the correct mitigation and is unchanged: every export carries provenance.json with the full environment of record.

5. VLM audits are a layer, not a gate. The frame audit found F-01, but its verdicts needed pixel analysis and introspection to become actionable - and one pre-fix verdict was a generous pass on an empty frame. The pixel assertion is what runs in the checks on every build; keep the vision model for qualitative regressions, and never let it be the only instrument that looked.

The recommendation that sits above all five: keep the checks authoritative. run_checks.py is now the single command that exercises the instrument end to end - tests, variants, guards, presets, exports, frame content - and every number in this report is rebuildable by running it on the delivered tree.

Appendix A. Environment and Evidence Index

Every artefact this audit produced is retained under the delivery bundle's evidence tree, and every figure in this report is rebuildable from the persisted scripts. Table 7 maps the evidence so a reviewer can walk from any claim in the preceding sections to the file that proves it. The reproduction commands below regenerate the headline numbers on any machine with the delivered tree installed.

Evidence	Location (inside the delivery bundle)
Pristine 1.0.0 upload	evidence/original/ (byte-identical extraction of the zip)
Original run logs	evidence/runs/test_suite_original.txt, checks_original.txt
Rebuilt run logs	evidence/runs/checks_enterprise.txt (128 tests, 11 checks, ink figures)
Broken GUI frames (1.0.0)	evidence/frames_gui/ - ten-frame headless dump, series panels empty
Fixed GUI frames (1.1.0)	evidence/frames_gui_enterprise/ - same dump on the rebuilt tree
Introspection probes	evidence/diagnose_series/ - view-limit state capture and the isolated-panel figure
VLM judgments	evidence/vlm/ (18 frames, original audit) and evidence/vlm_enterprise/ (13 frames, re-audit)
Preset sweeps	evidence/preset_runs_enterprise/ - 11 presets, 12 artefacts each; and the 1.0.0 sweep in evidence/preset_runs/
F-02 crash artefacts	evidence/diverging_export_probe/ - the six artefacts that succeeded before the crash
Report figures	evidence/report_assets/ - charts, frame comparison, cover
Audit tooling	scripts/ - every probe, sweep and chart script used by this audit

Table 7. Evidence index. All paths are relative to the delivery bundle root.

```
Reproduction
# reproduce the headline numbers on the delivered tree
python -m unittest discover -s tests -v      # 128 tests
python run_checks.py                         # 11 checks + ink assertion
ruff check . && mypy .                       # both gates at zero
pip install -e . && spobs --help             # package + entry point
```

The audit of record is AUDIT.md at the bundle root; this document is its typeset rendering. The rebuilt codebase, its tests, gates, packaging and documentation form the remainder of the bundle, and the CHANGELOG narrates every user-visible change between 1.0.0 and 1.1.0, including the record-hash break declared in section 7.